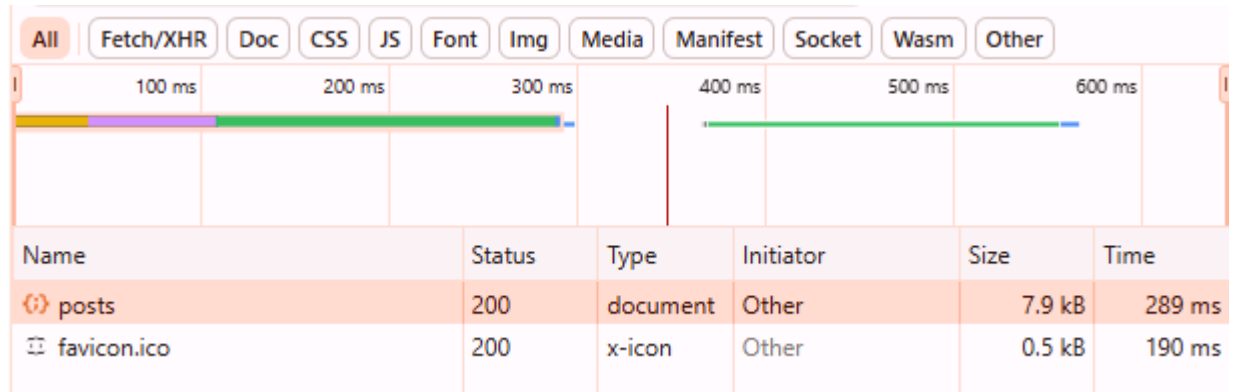


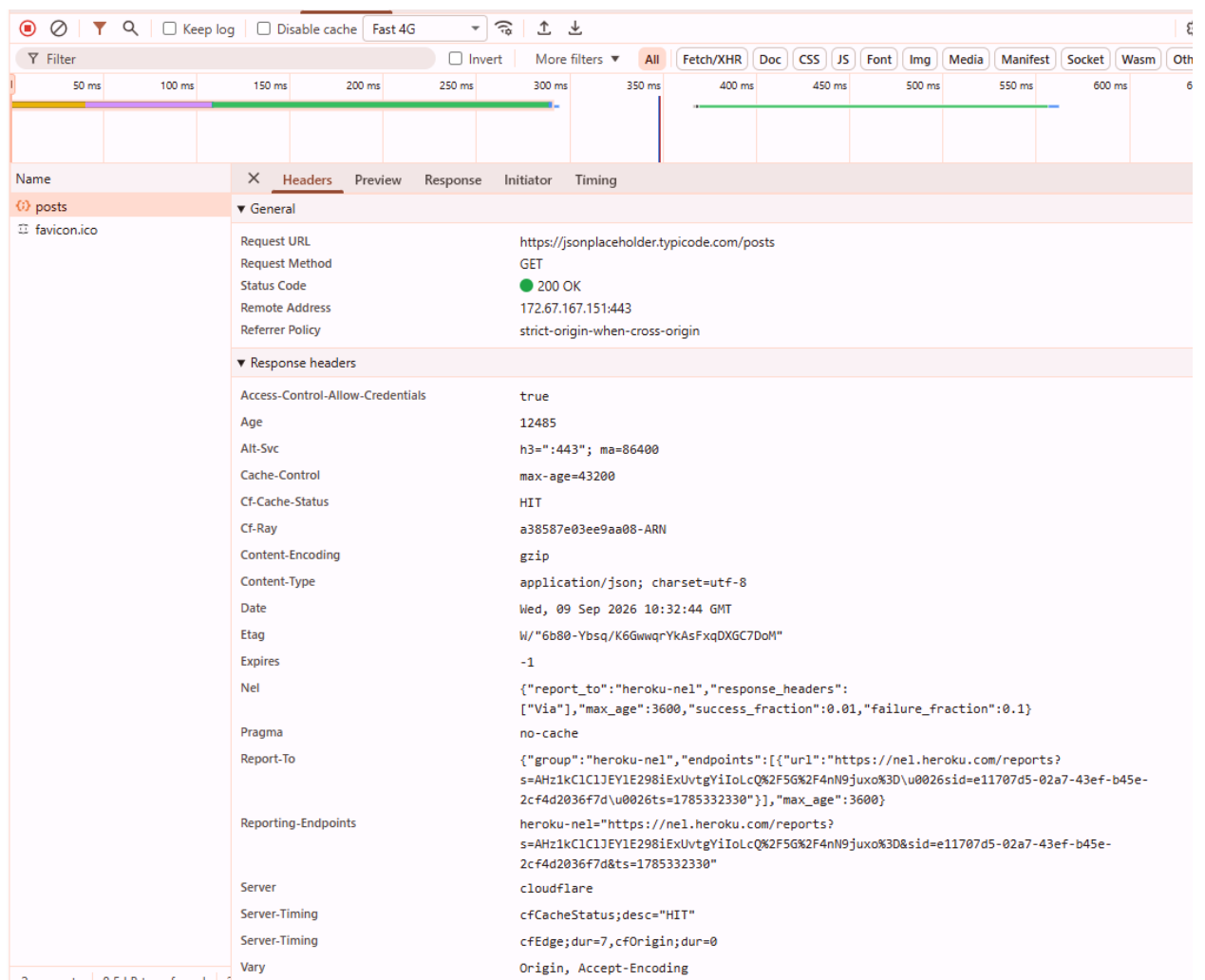
Практическая работа №1

2.2



2.3

Запрос:



Метод: GET

URL: <https://jsonplaceholder.typicode.com/posts>

Параметры: у данного запроса нет параметров.

Заголовки (Request Headers / Response Headers):

Response Headers:

▼ Response headers	
Access-Control-Allow-Credentials	true
Age	12485
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	a38587e03ee9aa08-ARN
Content-Encoding	gzip
Content-Type	application/json; charset=utf-8
Date	Wed, 09 Sep 2026 10:32:44 GMT
Etag	W/"6b80-Ybsq/K6GwwqrYkAsFqxqDXGC7DoM"
Expires	-1
Nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Pragma	no-cache
Report-To	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=AHz1kC1C1J1EY1E298iExUvtgViIoLcQ%2F5G%2F4nN9juxo%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1785332330"}],"max_age":3600}
Reporting-Endpoints	heroku-nel="https://nel.heroku.com/reports?s=AHz1kC1C1J1EY1E298iExUvtgViIoLcQ%2F5G%2F4nN9juxo%3D&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1785332330"
Server	cloudflare
Server-Timing	cfCacheStatus;desc="HIT"
Server-Timing	cfEdge;dur=7,cfOrigin;dur=0
Vary	Origin, Accept-Encoding
Via	2.0 heroku-router
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	1000
X-Ratelimit-Remaining	999
X-Ratelimit-Reset	1785332363

Request Headers:

▼ Request Headers	
:authority	jsonplaceholder.typicode.com
:method	GET
:path	/posts
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7
Priority	u=0, i
Sec-Ch-Ua	"Chromium";v="152", "Not?A_Brand";v="24", "Google Chrome";v="152"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Windows"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36

Тело ответа (Response / Preview):

Response:

X	Headers	Preview	Response	Initiator	Timing
1			[		
2			{		
3			"userId": 1,		
4			"id": 1,		
5			"title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",		
6			"body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas		
7			},		
8			{		
9			"userId": 1,		
10			"id": 2,		
11			"title": "qui est esse",		
12			"body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditi		
13			},		
14			{		
15			"userId": 1,		
16			"id": 3,		
17			"title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",		
18			"body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium		
19			},		
20			{		
21			"userId": 1,		
22			"id": 4,		
23			"title": "eum et est occaecati",		
24			"body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic		
25			},		
26			{		
27			"userId": 1,		
28			"id": 5,		
29			"title": "nesciunt quas odio",		
30			"body": "repudiandae veniam quaerat sunt sed\nalias aut fugiat sit autem sed est\nvoluptatem omnis possimus esse		
31			},		
32			{		
33			"userId": 1,		
34			"id": 6,		
35			"title": "dolorem eum magni eos aperiam quia",		

Preview:

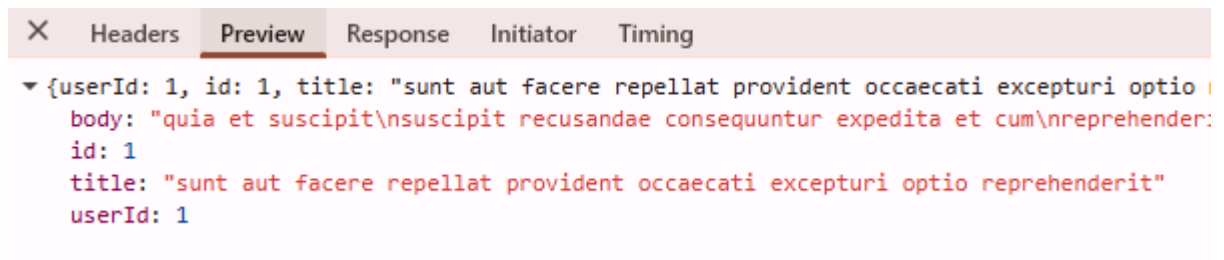
Метод: GET

URL: <https://jsonplaceholder.typicode.com/posts/1>

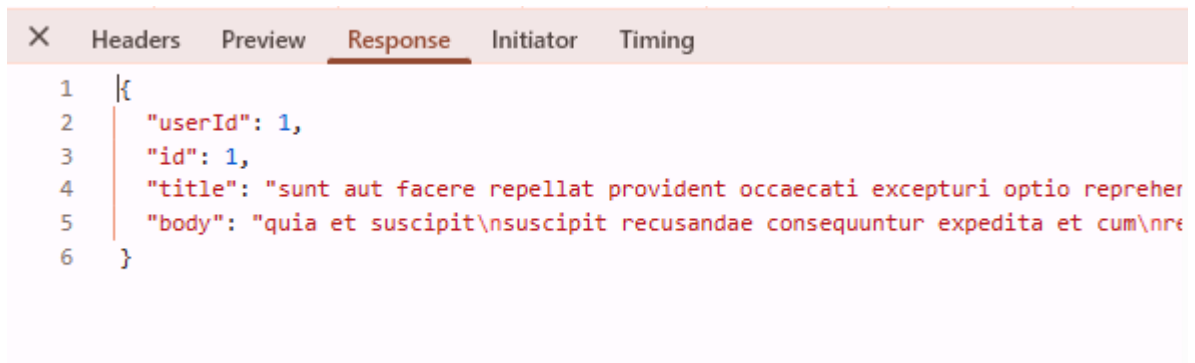
Практически ничего не меняется, кроме Response / Preview и времени выполнения.

Тело ответа.

Preview:



Response:



Код ответа: Status Code 200 OK

Время выполнения: Started at 0.87 ms

Количество записей:

► [index.html:11](#)
{userId: 1, id: 1, title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', body: 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas tota\nrem aut distinctio\nvoluptatem doloribus consectetur ullam ut aperiam sed sed est est autem est qui dolor anim.'}

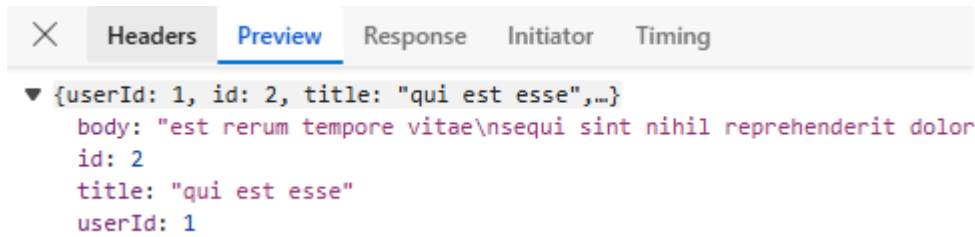
Другой запрос posts/2:

Name	×	Headers	Preview	Response	Initiator	Timing
index.html						
1						
2						
		▼ General				
		Request URL		https://jsonplaceholder.typicode.com/posts/2		
		Request Method		GET		
		Status Code		200 OK		
		Remote Address		172.67.167.151:443		
		Referrer Policy		strict-origin-when-cross-origin		

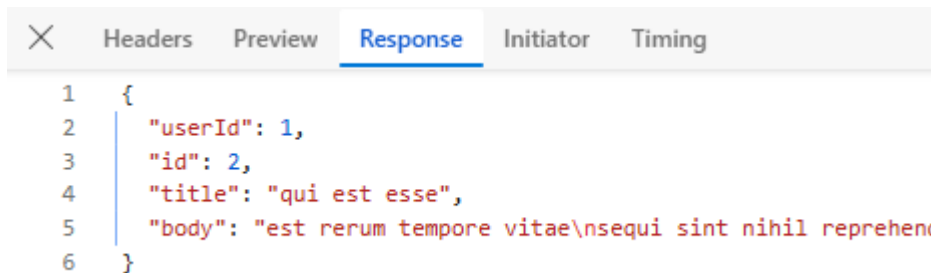
Метод: GET

URL: <https://jsonplaceholder.typicode.com/posts/2>

Preview:



Response:



Время выполнения: Started at 30.24 ms

Количество записей:

```
index.html:11
▶ {userId: 1, id: 2, title: 'qui est esse', body: 'est rerum tempore vitae\nequi sint nihil reprehendunt.aperiam non debitis possimus qui neque nisi nulla'}
```

2.5

Сравниваем:

<https://jsonplaceholder.typicode.com/posts>

Количество записей:

[illegible]

Size	0.8 kB
------	--------

Размер ответа:

Время загрузки: Started at 0.93 ms

<https://jsonplaceholder.typicode.com/posts?userId=1>

Количество записей: 10

Размер ответа:

Size
0.1 kB

Время загрузки: Started at 0.80 ms

Вывод, как параметры помогают уменьшить объём данных:

Чем больше фильтрации и условий, тем меньше объём данных.

2.6

Открываем несколько URL одновременно:

/posts/1

Queued at 25.43 ms

Время: Started at 27.20 ms

/posts/2

Queued at 25.87 ms

Время: Started at 28.98 ms

/posts/3

Queued at 26.27 ms

Время: Started at 27.95 ms

Когда мы открываем три разных адреса, браузер запускает их практически в одно и то же время. Это работает потому, что браузер может держать сразу несколько соединений с одним сайтом, общее время загрузки получается намного меньше, чем если бы они шли друг за другом.

Запросы избыточны. Если нам нужны три поста, логичнее сделать один запрос, который вернёт все три, а не отправлять три отдельных, это быстрее и эффективнее.

2.7

Headers:

Content-Type показывает, в каком формате пришли данные. Обычно это application/json, значит сервер вернул данные в формате JSON.

Cache-Control говорит браузеру, можно ли сохранять ответ и на сколько. no-cache означает, что каждый раз нужно проверять, не изменились ли данные на сервере.

ETag уникальный идентификатор версии ответа. Если данные на сервере не менялись, ETag остаётся тем же, и браузер может не загружать их заново, а взять из кэша.

Date дата и время, когда сервер отправил ответ. Помогает понять, когда данные были получены.

Server информация о том, какой сервер обработал запрос, полезно для диагностики, но пользователю обычно не важно.

Сравнение вкладок Preview и Response:

The image shows two screenshots of a web developer tool interface, specifically the 'Preview' and 'Response' tabs.

Preview Tab: The top screenshot shows the 'Preview' tab selected. It displays a list of 10 objects, each with a key-value pair for 'userId', 'id', and 'title'. The objects are numbered 0 through 9. The 'title' values are truncated with '...'.

Response Tab: The bottom screenshot shows the 'Response' tab selected. It displays the raw JSON data for the first three objects. The JSON is formatted with line numbers 1 through 19 on the left. The first object is an array containing three objects. The second object is a single object. The third object is a single object.

Preview показывает данные в удобном отформатированном виде. Можно раскрывать и сворачивать объекты, видно структуру данных. Подходит для быстрого просмотра содержимого ответа, особенно когда данных много.

Response показывает сырые данные точно в том виде, в котором их передал сервер. Без форматирования и подсветки. Нужен, когда хочется увидеть точный ответ сервера, проверить, нет ли лишних пробелов или скрытых символов.

Когда что использовать:

Response подходит, когда нужно увидеть, что именно прислал сервер в чистом виде, проверить, нет ли ошибок в ответе, посмотреть на нестандартные форматы данных. Preview удобен для быстрого просмотра содержимого, когда нужно найти нужное поле, проверить структуру ответа или просто понять, что пришло, не вникая в сырой текст.

2.9

GET /posts

The screenshot shows a REST client interface with a 'New Request' header. The method is 'GET' and the URL is 'https://jsonplaceholder.typicode.com/posts'. The response status is '200 OK' with a time of '1.21 s' and a size of '28 KB'. The response body is displayed in JSON format, showing an array of two post objects. The first object has 'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', and 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto'. The second object has 'userId': 1, 'id': 2, 'title': 'qui est esse', and 'body': 'est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla'.

```
1 [
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7   },
8   {
9     "userId": 1,
10    "id": 2,
11    "title": "qui est esse",
12    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
13  },
14 ]
```

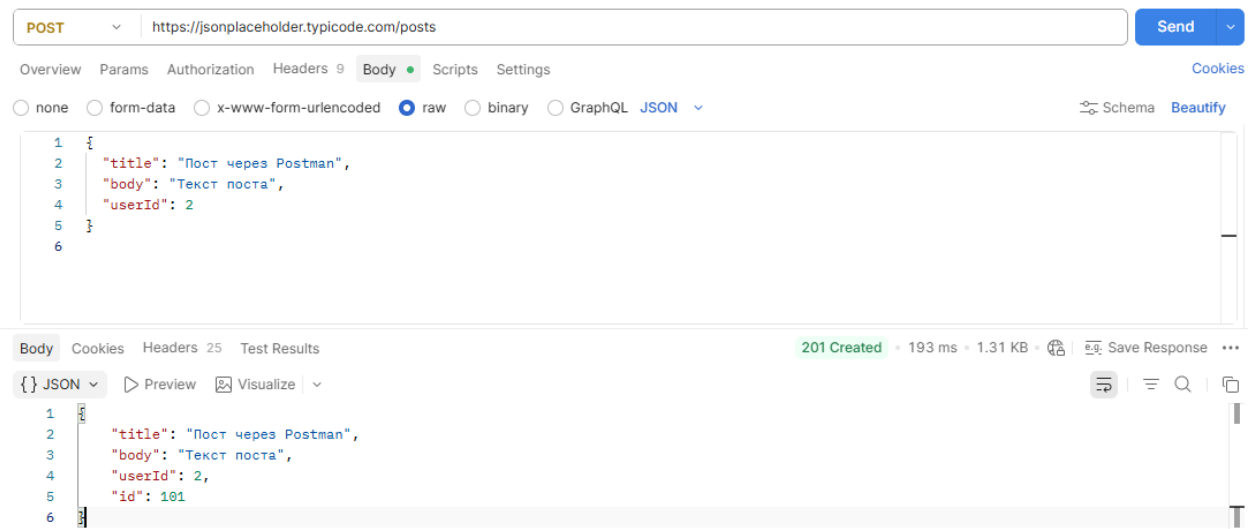
GET /posts/1

The screenshot shows a REST client interface with a 'New Request' header. The method is 'GET' and the URL is 'https://jsonplaceholder.typicode.com/posts/1'. The response status is '200 OK' with a time of '188 ms' and a size of '1.41 KB'. The response body is displayed in JSON format, showing a single post object. The object has 'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', and 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto'.

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

Сравнение ответов: в первом случае мы получили все 100 постов, а во втором только один.

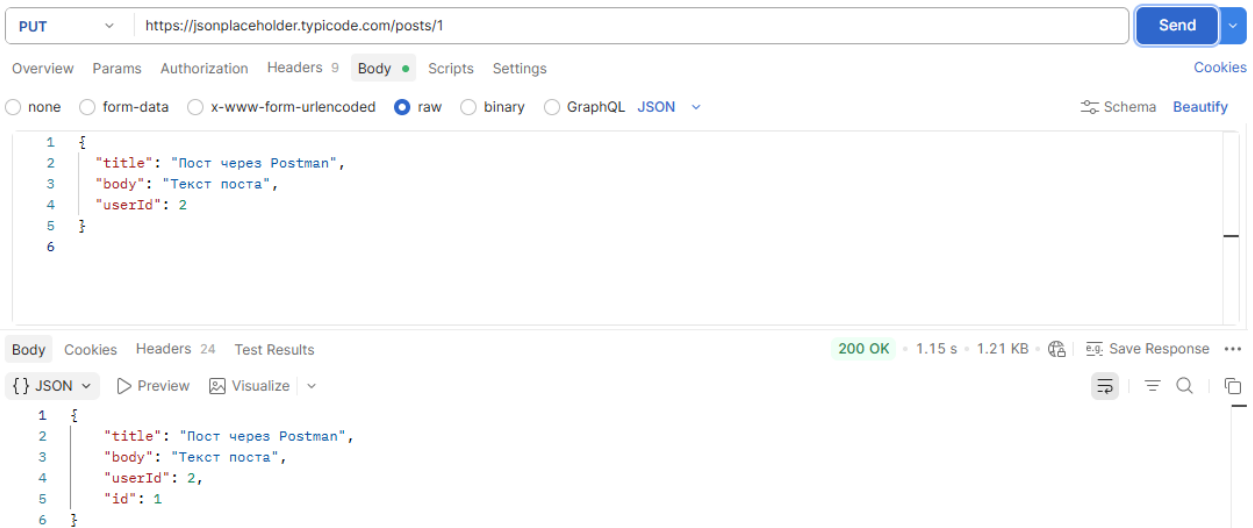
Выполнение POST /posts с JSON-телом:



В ответ сервер вернул созданный объект с присвоенным идентификатором, код ответа – 201 Created.

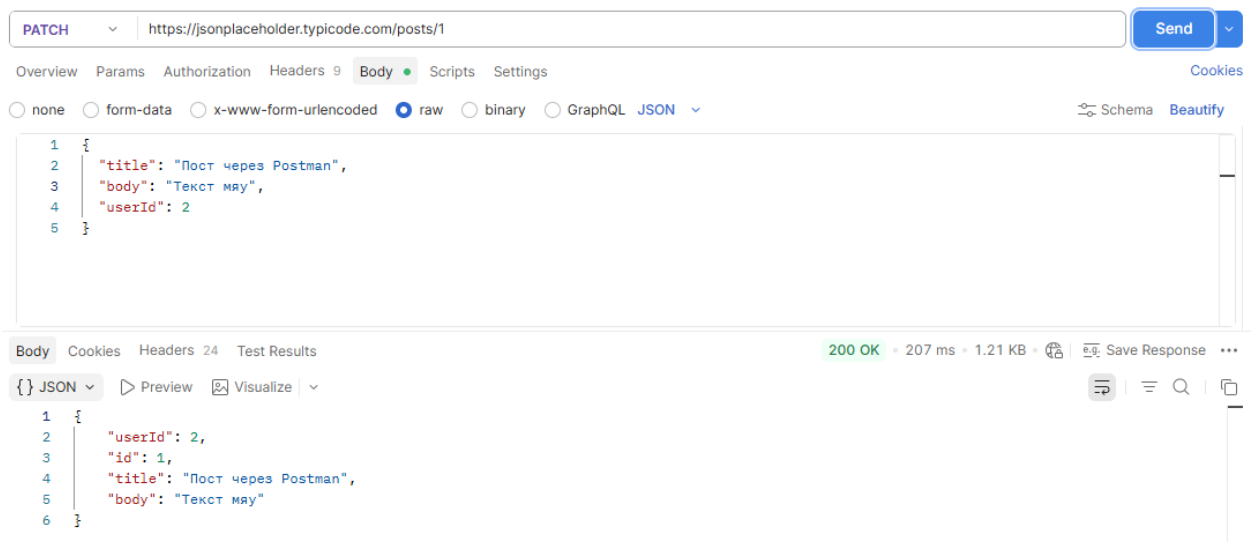
Выполнение:

PUT /posts/1 — полное обновление ресурса:



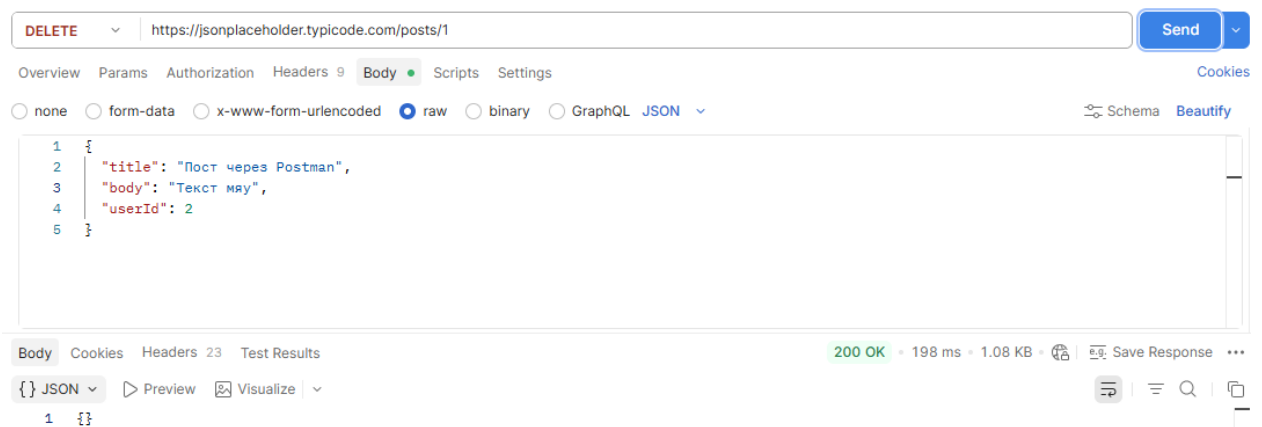
Сервер вернул обновлённый объект с кодом 200 OK. При PUT заменяется весь объект целиком.

PATCH /posts/1 — частичное обновление (изменить только один-два поля).



Сервер вернул обновлённый объект с кодом 200 OK. При PATCH меняются только те поля, которые мы указали, а остальные остаются без изменений.

Выполнение DELETE /posts/1 и зафиксируйте:



Сервер вернул обновлённый объект с кодом 200 OK. В данном API пришёл пустой ответ с кодом 200 OK. Тело ответа отсутствовало.

2.10

Сравнение DevTools и Postman

DevTools встроенный инструмент браузера, который показывает, что реально происходит на странице. Он автоматически перехватывает все запросы, которые отправляет сайт, когда человек нажимает на кнопки, открывает страницы, взаимодействует с интерфейсом, можно увидеть каждый запрос, его метод, URL, заголовки, тело ответа, код статуса и время

выполнения. Доступны вкладки для анализа производительности, памяти и безопасности. DevTools полезен, когда нужно проверить, почему страница загружается медленно, какие запросы отправляются при нажатии на кнопку, правильно ли работают кэширование и заголовки, когда нужно найти ошибку в работе сайта.

Postman подходит для ручной работы с API, можно самостоятельно создавать любые запросы, указывать метод, URL, заголовки и тело, а затем отправлять их на сервер и сразу видеть ответ. Запросы можно сохранять в коллекции, группировать по проектам, добавлять описания. В нем можно писать автотесты, проверять, правильно ли API обрабатывает разные данные, работать с авторизацией и токенами. Postman подходит для разработки API, когда нужно проверить, как сервер реагирует на разные данные, или отправить нестандартный запрос, лучше для автоматического тестирования.

	DevTools	Postman
Как попадают запросы	Автоматически перехватываются при работе с сайтом	Создаются вручную пользователем
Для чего нужен	Анализ сетевой активности живого сайта, проверка кэширования	Ручное тестирование API, разработка, автотесты, документирование
Можно ли сохранять запросы	Нет, они видны только в текущей сессии	Да, в коллекциях и папках
Поддерживает авторизацию	Да, но только в контексте страницы	Да
Можно ли писать тесты	Нет	Да, есть встроенные тесты и скрипты
Удобен для	Быстрой отладки и проверки сайта	Глубокого тестирования и разработки API

Контрольные вопросы

1. Что такое DevTools и какие задачи решают вкладки Elements, Console и Network?

DevTools – встроенные в браузер инструменты для разработчиков.

Elements показывает HTML-код страницы и CSS-стили.

Console выводит ошибки, предупреждения и логи. Также здесь можно выполнять JavaScript-код прямо на странице.

Network показывает все запросы, которые браузер отправляет на сервер. Можно увидеть метод, URL, заголовки, код ответа, размер данных и время загрузки.

2. Как открыть DevTools и какие настройки вкладки Network важны для замеров производительности?

DevTools открывается клавишей F12 или через меню браузера.

Network важные настройки:

Disable cache отключает кэш, чтобы видеть реальную загрузку без сохранённых копий.

Throttling можно выбрать профиль скорости и посмотреть, как страница загружается при медленном интернете.

Clear очистить список перед загрузкой страницы, чтобы не мешали старые запросы.

3. Перечислите основные части HTTP-запроса (метод, URL/параметры, заголовки, тело) и их назначение.

Метод показывает, что нужно сделать с данными (получить, создать, обновить или удалить).

URL и параметры - адрес ресурса на сервере и дополнительные данные в строке запроса.

Заголовки – служебная информация о запросе: формат данных, авторизация, настройки кэша и т.д.

Тело запроса – сами данные, которые отправляются на сервер.

4. Чем отличаются методы GET, POST, PUT, PATCH и DELETE? Приведите по одному практическому примеру.

GET – получить данные (открыть страницу со списком товаров).

POST – создать новый ресурс (зарегистрировать нового пользователя)

PUT – полностью обновить ресурс (полностью заменить профиль пользователя новыми данными).

PATCH – частично обновить ресурс (поменять только аватарку или телефон).

DELETE – удалить ресурс (удалить товар из корзины).

5. Для чего используются методы HEAD и OPTIONS и где встречается preflight-запрос?

HEAD – получает только заголовки ответа без тела. Используется, чтобы проверить, существует ли файл, какой у него размер и т.д.

OPTIONS – спрашивает сервер, какие методы и заголовки разрешены для данного ресурса.

Preflight-запрос – это запрос методом OPTIONS, который браузер автоматически отправляет перед сложными запросами (например, с нестандартными заголовками или с типом данных JSON). Проверяет, разрешит ли сервер такой запрос.

6. Расскажите о группах кодов ответов (1xx–5xx) и приведите примеры для 200, 301/302, 304, 404, 500.

1xx – информационные. 100 Continue – можно продолжать отправку данных.

2xx – успешные. 200 OK - всё хорошо, 201 Created – ресурс создан.

3xx – перенаправления. 301 Moved Permanently – ресурс перемещён навсегда, 302 Found – временно перемещён, 304 Not Modified – ничего не изменилось.

4xx – ошибки клиента. 400 Bad Request – неверный запрос, 404 Not Found – страница не найдена.

5xx – ошибки сервера. 500 Internal Server Error – что-то сломалось на сервере.

7. Что такое REST и как ресурс в REST представлен на уровне URL?

REST – это архитектурный подход к созданию API, где всё строится вокруг ресурсов (любой объект данных: пользователь, товар, пост и т.д.).

На уровне URL ресурсы выглядят:

/users — список всех пользователей

/users/5 — конкретный пользователь с ID=5

/users/5/posts — все посты пользователя с ID=5

Каждый URL — это адрес уникального ресурса.

8. Какие типы данных поддерживает JSON? Приведите короткий пример JSON-объекта и массива.

JSON поддерживает:

- строки: "Привет";
- числа: 42, 3.14;
- логические значения: true или false;
- объекты: {"имя": "Анна", "возраст": 25};
- массивы: ["яблоко", "банан", "апельсин"];
- значение null — пустота.

9. В чём отличие вкладок Preview и Response в карточке запроса DevTools?

Response показывает сырые данные в том виде, в котором их отправил сервер. Это чистый текст без форматирования.

Preview браузер красиво оформляет эти же данные, чтобы их было удобно читать, объекты можно раскрывать и сворачивать, видна структура.

10. Какие заголовки чаще всего анализируются при оптимизации (например, Content-Type, Cache-Control, ETag, Authorization)?

Content-Type — формат данных (например, JSON, HTML, картинка).

Cache-Control — настройки кэширования.

ETag — проверка изменений данных.

Authorization — токен для авторизации.

Date — время отправки ответа.

Server — тип сервера (помогает в диагностике).

11. Какие приёмы снижения нагрузки на сеть можно использовать для API: пагинация, фильтрация, кэширование и др.?

В пагинации данные выдаются по частям, например, по 10 или 20 записей на одну страницу, это позволяет не загружать сразу все записи, если их тысячи или миллионы.

В фильтрации сервер возвращает только те данные, которые действительно нужны пользователю. Например, можно запросить посты только одного юзера через параметр `?userId=1`.

В кэшировании часто используемые данные сохраняются на стороне клиента (в браузере) или на промежуточных серверах. При повторном запросе данные берутся из кэша, а не с основного сервера, что экономит трафик и ускоряет загрузку.

В уменьшении размера ответа вместо того, чтобы возвращать все поля объекта (например, полную информацию о пользователе со всеми полями), сервер может отдавать только самые нужные поля, исключая лишние.

С помощью правильных заголовков `Cache-Control` и `ETag` можно управлять кэшированием (например, указать, как долго хранить ответ, или проверять, изменились ли данные, прежде чем скачивать их заново).

С помощью сжатия данных сервер может сжимать ответ перед отправкой. Браузер или клиент получает сжатые данные и распаковывает их. Это сильно уменьшает объём трафика, особенно для текстовых данных вроде JSON или HTML.

12. Чем работа с API в Postman отличается от анализа запросов в DevTools, и когда уместен каждый инструмент?

DevTools хорош для анализа того, что реально происходит на странице в браузере. Можно увидеть все запросы, которые отправляет сайт, проверить время загрузки, кэширование и производительность. Используется, когда нужно понять, почему страница загружается медленно или что пошло не так.

Postman удобен для ручной работы с API, можно создавать любые запросы, менять заголовки и тело, сохранять запросы в коллекции, писать автотесты и работать с авторизацией. Используется, когда нужно

протестировать API отдельно от сайта, отправить нестандартный запрос или автоматизировать проверки.